

Classificação do tamanho do alvo a partir do movimento do mouse

1. Links

- Dados: <https://github.com/alangnclvs/mouse-pointing-tsc/tree/main/data>
- Código (Google Colab): https://colab.research.google.com/github/alangnclvs/mouse-pointing-tsc/blob/main/03_pointing_classification.ipynb

2. Problema e classes

Quis saber se é possível descobrir o tamanho de um alvo na tela só pelo jeito como o mouse se move até ele. Cada exemplo é um movimento: o cursor sai de um círculo no centro da tela e vai até um alvo que aparece a 300 px de distância. A série temporal é a velocidade do cursor durante esse trajeto, e a classe é o tamanho do alvo:

Classe	Diâmetro	Referência (WCAG 2.2)
pequeno	12 px	abaixo do tamanho mínimo
médio	24 px	tamanho mínimo (nível AA)
grande	44 px	tamanho recomendado (nível AAA)

Escolhi os tamanhos com base na WCAG 2.2, a norma de acessibilidade para web, que define um tamanho mínimo para botões e links. A lei de Fitts, da área de interação humano-computador, diz que quanto menor o alvo, maior o tempo do movimento, porque a parte final exige mais correções. Se o movimento carregar essa informação, daria para encontrar elementos de interface difíceis de clicar usando dados reais de uso, sem perguntar nada ao usuário. O movimento do mouse também é usado em biometria comportamental, em sistemas antifraude, e em ferramentas de análise de usabilidade.

3. Coleta dos dados

Escrevi um script em Python (tkinter, só biblioteca padrão) que abre a tarefa em tela cheia. Em cada tentativa eu clico no círculo do centro, um alvo aparece a 300 px numa direção sorteada e eu clico nele o mais rápido possível sem errar. O script grava o tempo e a posição (x, y) do cursor a cada evento de movimento, desde o momento em que o alvo aparece até o clique que acerta o alvo. Se eu clico fora, o movimento continua sendo gravado e o erro é contado. O rótulo é salvo pelo próprio script, então não houve rotulação manual.

As condições ficaram fixas em todas as sessões: monitor de 2560 x 1080 com escala de 100%, o mesmo mouse, velocidade do ponteiro 10 e "precisão aprimorada do ponteiro" ligada (o script registra essas configurações em cada sessão). Cada sessão teve 30 alvos, 10 de cada tamanho, em ordem aleatória com semente fixa. As direções foram distribuídas igualmente entre os tamanhos, para que nenhuma direção ficasse associada a uma classe.

Fiz uma sessão de teste (sessão 0), que não entrou nos dados, e depois 6 sessões em horários diferentes nos dias 25 e 26/09/2026, com pelo menos 30 minutos entre elas. Anotei como eu estava em cada uma:

Sessão	Conjunto	Data e hora	Condição	Cliques errados
1	treino	25/09 16:23	fim de um dia longo de trabalho, com sono e sem almoço	1
2	teste	25/09 16:59	durante uma reunião, só ouvindo, cansado	2
3	treino	25/09 17:40	logo depois do almoço, em pausa do trabalho	2
4	teste	25/09 20:33	depois de jantar e caminhar	3
5	treino	26/09 14:53	logo depois de acordar, em jejum	14
6	teste	26/09 19:27	depois de caminhar 6,5 km no sol, cansado	5

Separei treino e teste por sessão: sessões ímpares no treino e pares no teste. Assim nenhum movimento de teste vem da mesma sessão de um movimento de treino, e o teste mede se o modelo funciona numa sessão nova. No total são 180 movimentos, 90 de treino e 90 de teste, com 30 por classe em cada conjunto. No repositório, os dados estão em `data/train/<classe>/` e `data/test/<classe>/`, com um CSV por movimento (colunas `t`, `x` e `y`), além dos arquivos brutos de cada sessão.

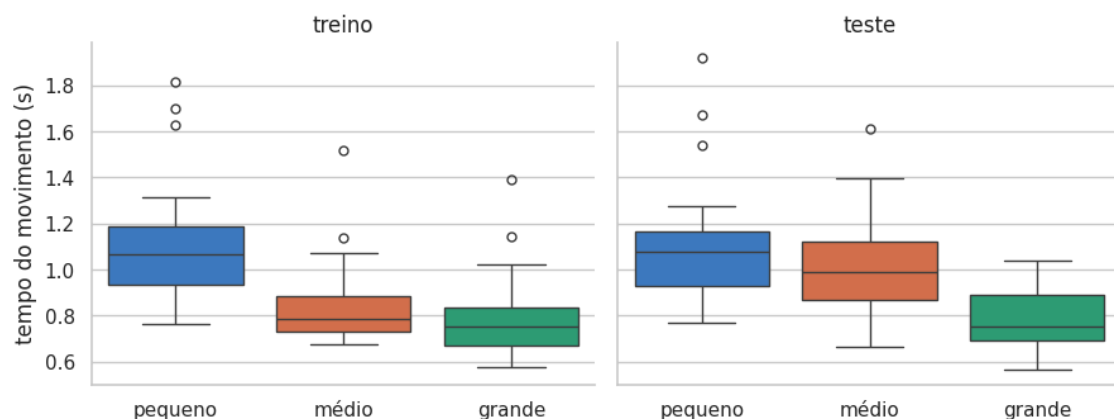


Figura 1. Tempo do movimento por classe no treino e no teste.

4. Pré-processamento

1. **Segmentação:** feita na própria coleta, já que cada arquivo é um movimento delimitado pelo aparecimento do alvo e pelo clique.
2. **Reamostragem:** o mouse só envia eventos quando se move (cerca de 8 ms entre eventos durante o movimento). Coloquei as posições numa grade regular de 100 Hz, repetindo a última posição enquanto o cursor está parado.
3. **Magnitude:** calculei a velocidade como a magnitude do deslocamento em x e y (px/s). Isso transforma a série bivariada em univariada e elimina o efeito da direção do alvo. Suavizei com uma média móvel de 50 ms.
4. **Trimming:** removi o tempo de reação do início. O movimento começa no primeiro ponto em que a velocidade passa de 10% do pico.
5. **Normalização:** a z-normalização de cada série foi tratada como parâmetro e testada na validação.

Depois disso, as séries ficaram com 37 a 160 pontos. Como os comprimentos são diferentes, usei métodos que aceitam séries de tamanhos diferentes. Nos dados de treino, o tempo do movimento seguiu a lei de Fitts: $\text{tempo} = 0,222 + 0,179 \times \text{ID}$, em que $\text{ID} = \log_2(D/W + 1)$, D é a distância e W a largura do alvo.

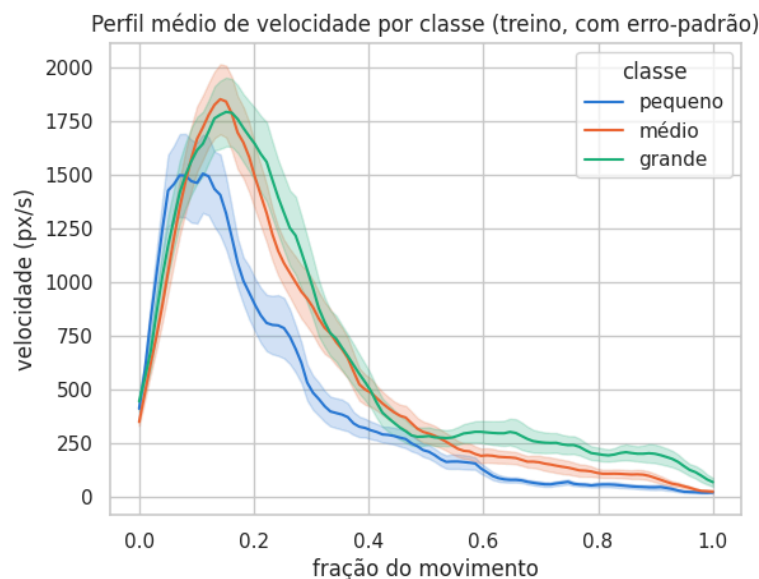


Figura 2. Perfil médio de velocidade por classe no treino, com o tempo normalizado entre o início do movimento (0) e o clique (1).

5. Protocolo experimental

Usei as sessões 1, 3 e 5 para treino e as sessões 2, 4 e 6 para teste. Todos os parâmetros foram escolhidos só com o treino, por validação cruzada deixando uma sessão de fora: treino com duas sessões e valido na terceira, três vezes. O critério foi a maior acurácia média na validação; nos empates, o menor desvio entre as sessões e depois a opção mais simples. O teste foi usado uma única vez, no final.

Modelo	Parâmetros avaliados	Escolhido	Acurácia na validação
1NN-DTW (baseline)	janela do DTW (0, 5, 10, 20 e 100%); série em px/s ou z-normalizada	z-normalizada, janela de 5%	46,7%
catch22 + Random Forest	características (só catch22; + média e desvio da velocidade; + duração); min_samples_leaf (1 ou 3)	catch22 + média, desvio e duração; min_samples_leaf = 3	57,8%

O catch22 foi calculado sobre a série z-normalizada, então descreve o formato do movimento. A média e o desvio da velocidade e a duração do movimento entraram como características extras. O Random Forest tem 500 árvores e semente fixa. Usei a biblioteca aeon (versão 1.6.0) para o DTW e o catch22, e o scikit-learn para o Random Forest.

6. Resultados

Modelo	Teste completo	Sessão 2	Sessão 4	Sessão 6
1NN-DTW (baseline)	34,4%	36,7%	23,3%	43,3%
catch22 + Random Forest	53,3%	63,3%	40,0%	56,7%

Com três classes balanceadas, o acerto ao acaso é de 33,3%. O Random Forest acertou 31 movimentos que o 1NN-DTW errou, e o contrário aconteceu em 14. O teste de McNemar deu $p = 0,016$, então a diferença entre os dois não parece ser por acaso. A acurácia do Random Forest no teste (53,3%) ficou perto da validação (57,8%), o que indica que a escolha de parâmetros não se ajustou demais ao treino.

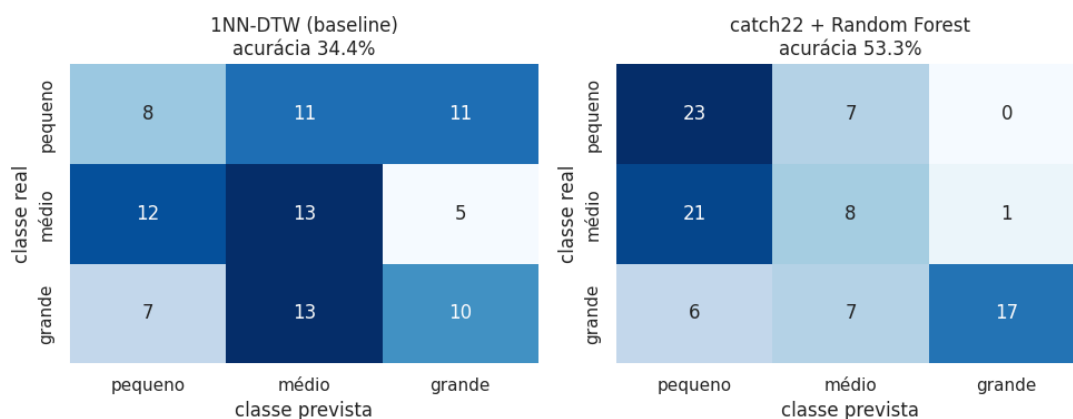


Figura 3. Matrizes de confusão no conjunto de teste.

No Random Forest, a classe grande teve a maior precisão (94%): quase tudo que ele chamou de grande era grande mesmo. A classe pequeno teve o maior recall (77%). O médio foi o problema: só 8 de 30 foram acertados, e 21 foram classificados como pequeno. No 1NN-DTW os erros ficaram espalhados por toda a matriz.

Análise dos erros

Olhando o tempo médio de cada sessão, o motivo dos erros no médio fica claro. Nas sessões de treino o movimento até o alvo médio levou de 0,75 a 0,90 s em média, perto do grande. Nas três

sessões de teste levou de 0,97 a 1,09 s, perto do pequeno (Figura 1). O modelo aprendeu que o médio é rápido e, no teste, os médios vieram mais lentos. Os erros do Random Forest também ficaram quase sempre entre tamanhos vizinhos: só 6 dos 42 erros confundiram pequeno com grande, contra 18 dos 59 erros do 1NN-DTW.

Para entender de onde vem o acerto, treinei o mesmo Random Forest só com algumas características. Só com a duração, a acurácia no teste foi de 38,9%. Com a média e o desvio da velocidade mais a duração, foi de 55,6%, e com todas as características, 53,3%. Ou seja, o que separa as classes é a combinação de duração com velocidade: nos alvos pequenos o movimento demora mais e é mais lento (Figuras 1 e 2). As características de formato do catch22 acrescentaram pouco, e as duas mais importantes no Random Forest foram justamente a duração e a velocidade média.

Isso explica o resultado do baseline. O 1NN-DTW escolhido usa a série z-normalizada, que tira a informação de velocidade, e o DTW alinha as séries esticando o tempo, o que compensa as diferenças de duração. Sobra só o formato da curva, que é parecido entre as classes, principalmente entre médio e grande (Figura 2). Na validação, a versão em px/s também não passou de 42,2%.

Por fim, os 9 movimentos de teste em que errei o clique antes de acertar o alvo foram os mais difíceis: o Random Forest acertou 33,3% deles, contra 55,6% dos demais.

7. Limitações

- Só eu participei, então o modelo aprendeu o meu jeito de usar o mouse e não dá para dizer que ele funciona para outras pessoas.
- Usei uma única distância (300 px), um mouse, um monitor e uma configuração de ponteiro. Com outra distância, o tempo do movimento muda mesmo com o mesmo alvo, e o modelo provavelmente erraria mais.
- Meu estado mudou bastante entre as sessões. Registrei, mas não controlei, e a mudança do alvo médio entre treino e teste mostra que essa variação afeta o resultado.
- Repeti a mesma tarefa em dois dias, então pode haver efeito de aprendizado. O resumo de tempos no fim de cada sessão também pode ter me incentivado a ir mais rápido nas seguintes.
- A sessão 5 teve bem mais cliques errados que as outras e ficou no treino, o que pode ter atrapalhado o que o modelo aprendeu sobre os alvos grandes.
- A tarefa é de laboratório. No uso normal o cursor não sai sempre do mesmo ponto e a pessoa não sabe que está sendo medida.
- Com 90 exemplos de teste, cada erro muda a acurácia em cerca de 1,1 ponto percentual, e a acurácia variou bastante entre as sessões de teste (de 40% a 63% no Random Forest). Diferenças pequenas, como entre 53,3% e 55,6%, podem ser acaso.